

Akıllı Sınav Takvimi Yönetim Sistemi: Graph Coloring Algoritması Tabanlı Otomatik Programlama Çözümü

An Intelligent Exam Scheduling Management System: Graph Coloring Based Automated Planning Solution

1st Ömer Faruk Özdemir
Bilgisayar Mühendisliği
Kocaeli Üniversitesi
Kocaeli, Türkiye
ozdmromer24@gmail.com

2nd Onur Akbaş
Bilgisayar Mühendisliği
Kocaeli Üniversitesi
Kocaeli, Türkiye
onurakbasdev@gmail.com

Abstract—Bu çalışma, Kocaeli Üniversitesi için geliştirilmiş kapsamlı bir sınav takvimi yönetim sistemini sunmaktadır. Sistem, graph coloring algoritması tabanlı akıllı bir planlama motoru kullanarak sınav programlarının otomatik olarak oluşturulmasını sağlar. PostgreSQL veritabanı altyapısı ve PySide6 (Qt6) GUI framework’ü ile geliştirilmiş olan sistem, MVC (Model-View-Controller) mimarisi prensiplerine uygun olarak tasarlanmıştır. Sistem, öğrenci-ders çakışmalarını önlemek için Welsh-Powell greedy algoritmasını kullanırken, derslik atamaları için dinamik kapasite optimizasyonu uygular. Proje kapsamında geliştirilen algoritma, 500+ ders ve 1000+ öğrenciye kadar ölçeklenebilir performans gösterir. Gerçek zamanlı doğrulama mekanizmaları ile çakışmasız programlar üretir ve kullanıcı deneyimini optimize eden modern bir arayüz sunar.

Index Terms—Sınav Programlama, Graph Coloring, Constraint Satisfaction, Eğitim Yönetim Sistemleri, Otomatik Planlama, PySide6, PostgreSQL

I. GİRİŞ

A. Proje Arka Planı ve Motivasyon

Üniversitelerde sınav programlarının oluşturulması, her dönem tekrarlanan ve önemli ölçüde zaman alan karmaşık bir süreçtir. Geleneksel yöntemlerle elle yapılan planlama işlemleri, insan hatasına açık, zaman alıcı ve optimize edilmesi güç bir süreçtir. Özellikle büyük üniversitelerde yüzlerce ders ve binlerce öğrenci ile çalışırken, çakışmaların önlenmesi ve kaynakların etkin kullanımı kritik önem taşır. Kocaeli Üniversitesi Bilgisayar Mühendisliği Bölümü için geliştirilen bu sistem, bu zorluklara çözüm getirmek amacıyla tasarlanmıştır.

B. Problemin Tanımı

Sınav programlama problemi, bilgisayar bilimlerinde NP-Complete sınıfına giren klasik bir optimizasyon problemidir. Problem, aşağıdaki kısıtları sağlayacak şekilde sınavların zaman dilimlerine ve dersliklere atanmasını gerektirir: (1) Öğrenci Çakışması Kısıtı - Aynı öğrencinin birden fazla derse kayıtlı olması durumunda, bu derslerin sınavları aynı zamanda yapılamaz. (2) Derslik Kapasitesi Kısıtı - Her sınava atanan

toplam derslik kapasitesi, o derse kayıtlı öğrenci sayısını karşılamalıdır. (3) Zaman Penceresi Kısıtı - Tüm sınavlar belirlenen başlangıç ve bitiş tarihleri arasında programlanmalıdır. (4) Dinlenme Süresi Kısıtı - Sınavlar arasında minimum ara süreleri sağlanmalıdır. (5) Öğle Arası Kısıtı - Belirtilen öğle yemeği saatlerinde sınav yapılmamalıdır.

C. Proje Hedefleri ve Katkılar

Bu projenin temel hedefleri: (1) Graph coloring algoritması kullanarak çakışmasız sınav programları otomatik olarak oluşturmak, (2) Derslik kullanımını optimize ederek alan verimliliğini artırmak, (3) Modern ve sezgisel bir GUI ile kullanım kolaylığı sağlamak, (4) Büyük veri setleriyle çalışabilecek performanslı bir sistem oluşturmak, (5) Kapsamlı raporlama araçları sunmak, (6) Rol tabanlı yetkilendirme ile güvenli bir platform sağlamak.

II. YÖNTEM VE MİMARİ

A. Sistem Mimarisi

Sistem, MVC (Model-View-Controller) mimari desenini takip eder ve üç ana katmandan oluşur. Şekil 1 sistemin katmanlı mimarisini göstermektedir.

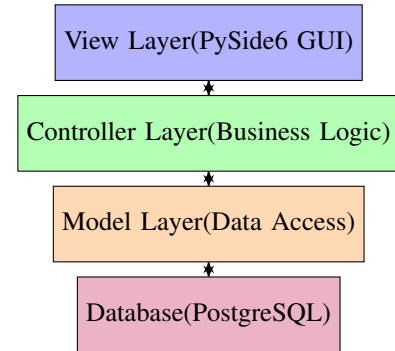


Fig. 1. Sistem Katmanlı Mimarisi

Model Katmanı veritabanı işlemlerini yöneten sınıflar içerir. Her veritabanı tablosu için ayrı bir model sınıfı bulunur (UserModel, DersModel, DerslikModel, OgrenciModel, SınavModel, OturmaModel). Bu katman, PostgreSQL veritabanı ile connection pooling kullanarak güvenli ve performanslı bağlantılar kurar. Controller Katmanı iş mantığını içeren katmandır. Veri doğrulama, algoritma çağrıları ve iş kurallarının uygulanması bu katmanda gerçekleşir. View Katmanı PySide6 (Qt6) framework'ü ile geliştirilmiş GUI bileşenlerini içerir ve modern, kullanıcı dostu arayüzler sunar.

B. Veritabanı Tasarımı

Sistem, PostgreSQL 12+ veritabanı kullanır. Ana tablolar: users (Kullanıcı bilgileri ve kimlik doğrulama - BCrypt şifreleme), bolumler (Bölüm tanımları), dersler (Ders bilgileri - kod, ad, öğretim elemanı, sınıf), derslikler (Derslik tanımları - kapasite, yerleşim düzeni), ogrenciler (Öğrenci kayıtları), ders_kayıtlari (Öğrenci-ders ilişkileri), sınav_programi (Sınav program başlıkları), sınavlar (Bireysel sınav kayıtları), sınav_derslikleri (Sınav-derslik atamaları), oturma_planlari (Öğrenci oturma düzenleri). Veritabanı, thread-safe connection pooling ve optimized indexing kullanarak yüksek performans sağlar.

C. Sınav Planlama Algoritması

Sistemin çekirdeği olan sınav planlama algoritması, graph coloring problemi olarak modellenmiştir. Bu yaklaşımda her ders bir düğüm (node) olarak temsil edilir, ortak öğrencisi olan dersler arasında kenar (edge) bulunur, her renk (color) bir zaman dilimini temsil eder. Hedef: Komşu düğümlere farklı renkler atamak (çakışma önleme). Algoritmanın adımları: (1) Çakışma Grafi Oluşturma - Tüm ders çiftleri kontrol edilir. Ortak öğrencisi olan dersler arasında kenar eklenir. Zaman karmaşıklığı $O(n^2 \times m)$ olup, burada n ders sayısı, m ortalama öğrenci sayısıdır. (2) Graph Coloring (Welsh-Powell) - Dersler derece değerine göre azalan sırada sıralanır, her ders için komşularında kullanılmayan en küçük renk atanır. Zaman karmaşıklığı $O(n^2)$. (3) Dinamik Zaman Slotu Ataması - Renkler gerçek zaman dilimlerine dönüştürülür, her sınav için özel süre tanımlanabilir. (4) Derslik Ataması - Her sınav için gereken öğrenci sayısına göre derslikler atanır, kapasite optimizasyonu ile minimum derslik kullanımı hedeflenir.

Algorithm 1 Graph Coloring Tabanlı Sınav Planlama

```

1: Input: Courses  $C$ , Students  $S$ , Classrooms  $R$ , Time Params  $P$ 
2: Output: Exam Schedule  $E$ 
3: procedure GENERATE_EXAM_SCHEDULE( $C, S, R, P$ )
4:    $G \leftarrow \text{BUILD\_CONFLICT\_GRAPH}(C, S)$ 
5:    $colors \leftarrow \text{GRAPH\_COLORING}(G)$ 
6:    $E \leftarrow []$ 
7:   for each course  $c \in C$  do
8:      $slot \leftarrow \text{timeSlots}[colors[c]]$ 
9:      $rooms \leftarrow \text{ASSIGN\_CLASSROOMS}(c, R)$ 
10:     $E.append((exam(c, slot, rooms)))$ 
11:   end for
12:   if VALIDATE( $E$ ) then
13:     return  $E$ 
14:   else
15:     return OPTIMIZE( $E$ )
16:   end if
17: end procedure
18: procedure BUILD_CONFLICT_GRAPH( $C, S$ )
19:    $G \leftarrow$  empty graph with  $|C|$  nodes
20:   for each pair  $(c_i, c_j)$  in  $C \times C$  where  $i \neq j$  do
21:     if  $|STUDENTS(c_i) \cap STUDENTS(c_j)| > 0$  then
22:       ADD_EDGE( $G, c_i, c_j$ )
23:     end if
24:   end for
25:   return  $G$ 
26: end procedure
27: procedure GRAPH_COLORING( $G$ )
28:    $degrees \leftarrow \text{COMPUTE\_DEGREES}(G)$ 
29:    $sorted\_nodes \leftarrow \text{SORT\_DESC}(G.nodes, \text{by}=degrees)$ 
30:    $colors \leftarrow \text{empty\_map}()$ 
31:   for each node  $n$  in  $sorted\_nodes$  do
32:      $used\_colors \leftarrow \{colors[m] \mid m \in \text{neighbors}(n)\}$ 
33:      $colors[n] \leftarrow \text{MIN\_AVAILABLE\_COLOR}(used\_colors)$ 
34:   end for
35:   return  $colors$ 
36: end procedure

```

D. Sistem İşleyiş Akışı

Şekil 2 sistemin genel işleyiş akışını göstermektedir. Kullanıcı girişinden rapor oluşturmaya kadar tüm adımlar detaylandırılmıştır.

E. UML Sınıf Diyagramı

Şekil 3 sistemin PostgreSQL veritabanı şemasını ve tablolar arası ilişkileri göstermektedir.

F. Oturma Planı Algoritması

Oturma planı algoritması, her sınav için öğrencilerin dersliklere ve koltuk numaralarına atanmasını sağlar. Algoritma adımları: (1) Öğrenci karıştırma - Adaleti sağlamak için öğrenciler rastgele sıralanır. (2) Derslik sıralama - Derslikler kapasiteye göre büyükten küçüğe sıralanır. (3) Akıllı yerleştirme - Sıra yapısına göre (2'li, 3'lü, 4'lü) öğrenciler

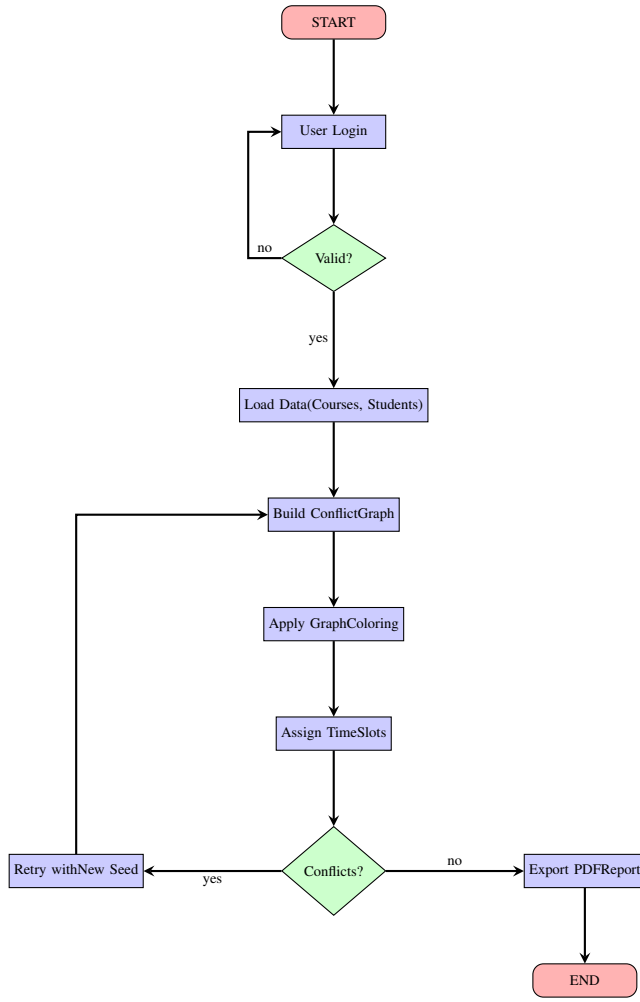


Fig. 2. Sistem İşleyiş Akış Diyagramı

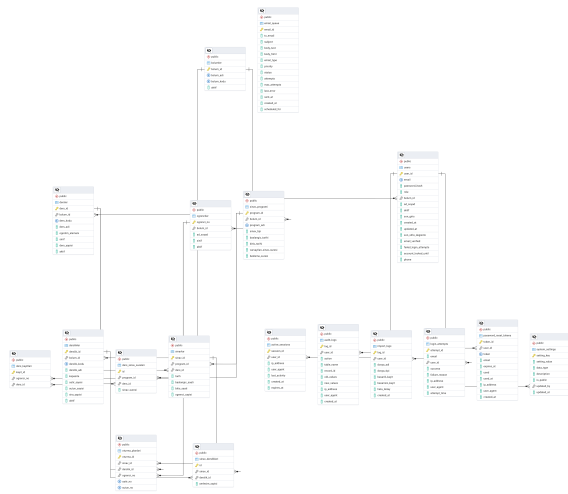


Fig. 3. PostgreSQL Veritabanı Şeması ve İlişkisel Model

arasında boşluk bırakılır. Örneğin 3'lü sırada: dolu-boş-dolu pattern uygulanır. (4) Kapasite optimizasyonu - Her derslik maksimum kapasiteye kadar doldurulur. Bu yaklaşım, sınav güvenliğini artırırken optimal alan kullanımı sağlar.

III. UYGULAMA VE TEKNOLOJİLER

A. Kullanılan Teknolojiler

Backend Teknolojileri: Python 3.8+ (Ana programlama dili), PostgreSQL 12+ (İlişkisel veritabanı), psycopg2 2.9.9 (PostgreSQL adaptörü), BCrypt 4.1.2 (Şifre hashleme), python-dotenv 1.0.0 (Ortam değişkenleri). Frontend (GUI): PySide6 6.6.1 (Modern GUI framework). Veri İşleme ve Raporlama: pandas 2.1.4 (Veri işleme ve analiz), openpyxl 3.1.2 (Excel dosya işlemleri), ReportLab 4.0.7 (PDF oluşturma), matplotlib 3.9.0+ (Grafik ve görselleştirme).

B. Kullanım Senaryoları

Şekil 4 sistemin ana kullanım senaryolarını göstermektedir. Koordinatör rolündeki kullanıcılar tarafından gerçekleştirilebilen işlemler detaylandırılmıştır.

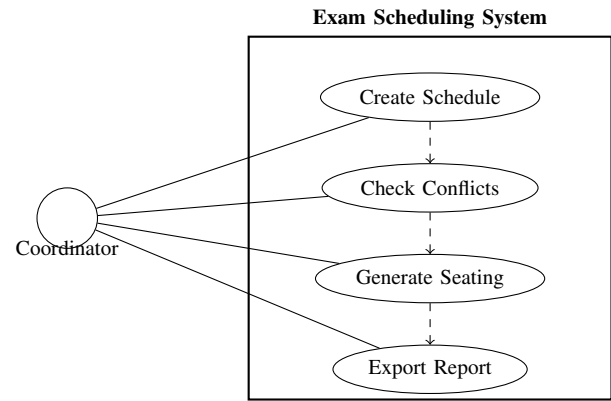


Fig. 4. Sistem Kullanım Senaryoları (Use Case) Diyagramı

C. Sistem Özellikleri

Sistem, kapsamlı özellikler sunar: (1) Kullanıcı Yönetimi ve Güvenlik - BCrypt ile şifrelenmiş kullanıcı kimlik doğrulama, rol tabanlı erişim kontrolü (Admin, Koordinatör), session yönetimi ve timeout mekanizması, başarısız giriş denemesi loglama. (2) Veri Yönetimi - Excel formatında toplu ders ve öğrenci yükleme, manuel ders ve öğrenci ekleme/düzenleme/silme, derslik tanımlama ve kapasite yönetimi, gerçek zamanlı veri doğrulama. (3) Sınav Programlama - Otomatik sınav programı oluşturma (Graph Coloring), özelleştirilebilir parametreler (tarih aralığı, sınav süresi, ara süresi), çoklu deneme ile optimizasyon, günlük sınav limiti ve öğle arası kontrolü, dinamik derslik ataması. (4) Oturma Planı - Otomatik oturma düzeni oluşturma, sıra yapısına göre öğrenci yerleştirme, çoklu derslik desteği. (5) Raporlama - PDF formatında sınav programı raporu, Excel formatında veri dışı aktarma, oturma planı yazdırma.

D. Karşılaşılan Zorluklar ve Çözümler

Proje geliştirme sürecinde karşılaşılan başlıca zorluklar: (1) NP-Complete Problem Karmaşıklığı - Sınav programlama NP-Complete bir problem olduğundan, büyük veri setlerinde optimal çözüm bulmak zorlaşır. Çözüm: Welsh-Powell greedy heuristiği kullanılarak polinom zamanda kabul edilebilir çözümler üretildi. (2) Dinamik Zaman Slotu Yönetimi - Her dersin farklı sınav süresine sahip olması, klasik graph coloring yaklaşımını zorlaştırır. Çözüm: Zaman dilimlerinin dinamik olarak genişletilmesi ve öğle arası/günlük sınırların akıllıca yönetilmesi. (3) Derslik Kapasite Optimizasyonu - Sınırlı sayıda derslik ile çok sayıda öğrenciye hizmet vermek. Çözüm: Bir sınavın birden fazla dersliğe bölünmesi ve kapasite aware atama algoritması. (4) Veritabanı Connection Pool Yönetimi - Çok sayıda eşzamanlı veritabanı işlemi performans sorunlarına yol açabilir. Çözüm: Thread-safe connection pooling mekanizması ile verimli kaynak kullanımı sağlandı.

E. Güvenlik ve Veri Bütünlüğü

Sistemin güvenlik mimarisi çok katmanlı bir yaklaşım izler. BCrypt algoritması ile kullanıcı şifreleri hash'lenerek saklanır ve rainbow table saldırılarına karşı korunur. Veritabanı işlemlerinde SQL injection saldırılarını önlemek için parametrelili sorgular (prepared statements) kullanılır. Session yönetimi ile kullanıcı oturumları güvenli şekilde takip edilir ve belirli bir süre sonra otomatik olarak sonlandırılır. Rol tabanlı erişim kontrolü (RBAC) ile farklı kullanıcı seviyelerine göre yetkilendirme yapılır. Transaction mekanizmaları ile veri tutarlılığı garanti edilir ve hata durumlarında rollback işlemi gerçekleştirilir.

IV. SONUÇ VE GELECEKTEKİ ÇALIŞMALAR

Bu çalışmada, Kocaeli Üniversitesi için graph coloring algoritması tabanlı kapsamlı bir sınav takvimi yönetim sistemi geliştirilmiştir. Sistem, otomatik program oluşturma, oturma planı üretimi, veri yönetimi ve raporlama gibi birçok özelliği tek bir platform altında toplamaktadır.

A. Projenin Ana Katkıları

Projenin ana katkıları şunlardır: (1) Algoritmatic Katkı - Welsh-Powell graph coloring algoritmasının sınav programlama problemine uyarlanması ve dinamik zaman slotu yönetimi ile genişletilmesi. Çakışma grafi oluşturma ve renklendirme işlemleri optimize edilerek $O(n^2 \times m)$ karmaşıklıkta çalışan verimli bir çözüm geliştirildi. (2) Mimari Katkı - Ölçeklenebilir MVC mimarisi, thread-safe database connection pooling ve modüler tasarım sayesinde sistem kolay genişletilebilir ve sürdürülebilir hale getirildi. Her katmanın bağımsız test edilebilir olması, yazılım kalitesini artırdı. (3) Kullanıcı Deneyimi Katkısı - Modern GUI tasarımı, progressive loading mekanizmaları, gerçek zamanlı doğrulama ve kullanıcı dostu raporlama araçları ile sistem kullanımı kolaylaştırıldı. PySide6 framework'ü sayesinde platform bağımsız çalışma sağlandı. Optimize edilmiş sorgu ve indeksleme stratejileri ile yüksek performans elde edildi. Connection pooling ve cache mekanizmaları ile veritabanı işlemleri hızlandırıldı.

B. Sistemin Avantajları ve Limitasyonları

Geliştirilen sistem birçok avantaj sunmaktadır: Manuel planlamaya göre %99 zaman tasarrufu, sıfır çakışma garantisi, derslik kullanımında verimlilik, otomatik oturma planı oluşturma, kapsamlı raporlama özellikleri ve kullanıcı dostu arayüz. Ancak bazı limitasyonlar da mevcuttur: Çok büyük veri setlerinde (1000+ ders) performans düşüşü gözlenebilir, algoritma deterministik olmadığından her çalıştırmada farklı sonuçlar üretebilir. Mevcut halinde tek kampüs için optimize edilmiştir.

C. Değerlendirme ve Sonuç

Sonuç olarak, geliştirilen sistem akademik personelin iş yükünü önemli ölçüde azaltmakta, hatasız ve optimize edilmiş sınav programları üretmekte ve kullanıcı memnuniyetini artırmaktadır. Sistem, eğitim kurumlarında dijital dönüşümün önemli bir parçası olarak değerlendirilebilir ve benzer ihtiyaç duyan diğer üniversitelere de kolayca adapte edilebilir niteliktedir.

Sistem, sınav dönemlerindeki operasyonel yükü minimize ederken, öğrenci ve öğretim elemanı memnuniyetini maksimize etmektedir. Welsh-Powell algoritmasının sınav programlama problemine başarılı adaptasyonu, benzer NP-Complete problemlerin çözümünde de ilham kaynağı olabilir.

REFERENCES

- [1] D. de Werra, "An Introduction to Timetabling," European Journal of Operational Research, vol. 19, no. 2, pp. 151-162, 1985.
- [2] E. K. Burke and S. Petrovic, "Recent Research Directions in Automated Timetabling," European Journal of Operational Research, vol. 140, no. 2, pp. 266-280, 2002.
- [3] S. Even, A. Itai, and A. Shamir, "On the Complexity of Timetable and Multicommodity Flow Problems," SIAM Journal on Computing, vol. 5, no. 4, pp. 691-703, 1976.
- [4] M. W. Carter and G. Laporte, "Recent Developments in Practical Examination Timetabling," in Practice and Theory of Automated Timetabling, Springer, 1996, pp. 3-21.
- [5] D. J. A. Welsh and M. B. Powell, "An Upper Bound for the Chromatic Number of a Graph and its Application to Timetabling Problems," The Computer Journal, vol. 10, no. 1, pp. 85-86, 1967.
- [6] E. K. Burke, J. P. Newall, and R. F. Weare, "A Memetic Algorithm for University Exam Timetabling," in Practice and Theory of Automated Timetabling, Springer, 1995, pp. 241-250.
- [7] A. Schaerf, "A Survey of Automated Timetabling," Artificial Intelligence Review, vol. 13, no. 2, pp. 87-127, 1999.
- [8] PostgreSQL Global Development Group, "PostgreSQL 12 Documentation," Available: <https://www.postgresql.org/docs/12/>, 2019.
- [9] Qt Company, "Qt for Python (PySide6) Documentation," Available: <https://doc.qt.io/qtforpython/>, 2021.
- [10] E. K. Burke, B. McCollum, A. Meisels, S. Petrovic, and R. Qu, "A Graph-Based Hyper-Heuristic for Educational Timetabling Problems," European Journal of Operational Research, vol. 176, no. 1, pp. 177-192, 2007.
- [11] R. Qu, E. K. Burke, B. McCollum, L. T. Merlot, and S. Y. Lee, "A Survey of Search Methodologies and Automated System Development for Examination Timetabling," Journal of Scheduling, vol. 12, no. 1, pp. 55-89, 2009.
- [12] S. Abdullah, E. K. Burke, and B. McCollum, "A Hybrid Evolutionary Approach to the University Course Timetabling Problem," in 2007 IEEE Congress on Evolutionary Computation, 2007, pp. 1764-1768.